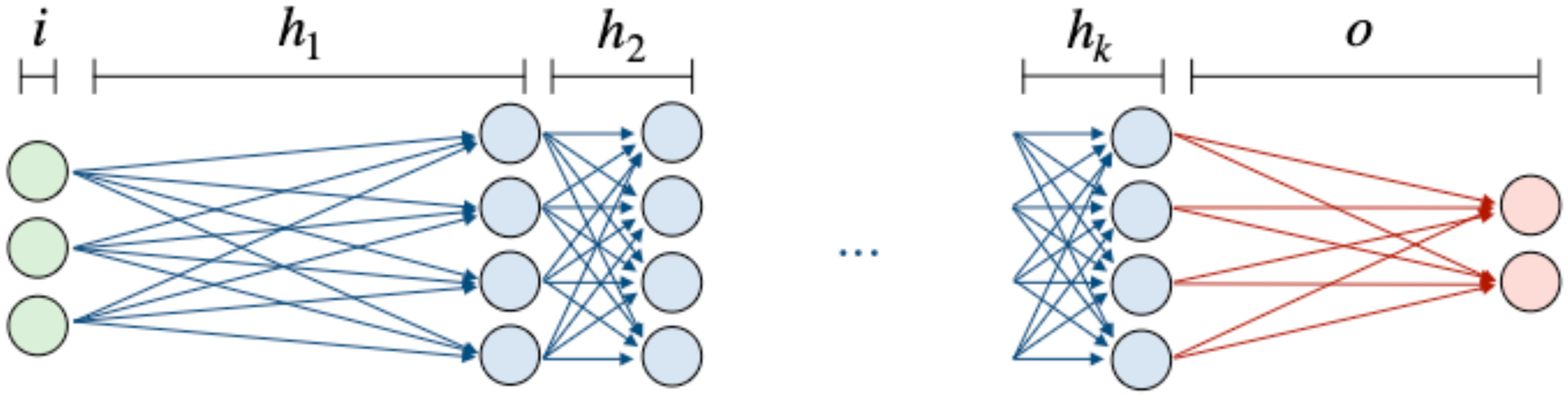
AIX

Multi-Layer Perceptron (MLP)

Mingyu Kim,
Mar, 12nd, 2026

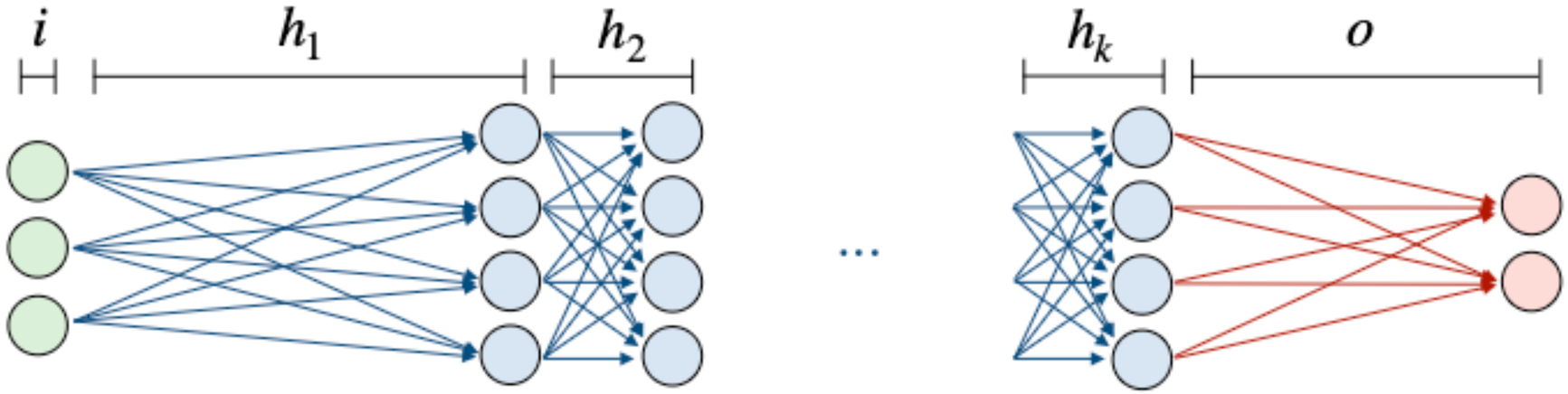
Feed-Forward Neural Network

- Network structure: input/hidden/output, activations, softmax.
  - Input layer i: It receives the input as a vector of numbers.
  - Hidden layers $h_1, \ldots, h_k$: They transform the input using multiplications, additions and nonlinearities.
  - Output layer o: It outputs a final vector that represents the model prediction.



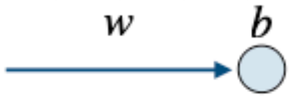
Feed-Forward Neural Network

- Neural nets learn complex patterns via connected units + nonlinearities.
- A feed-forward NN (FFNN) flows input → hidden layers → output (one direction).
- The output is a prediction vector o; interpretation depends on the task.

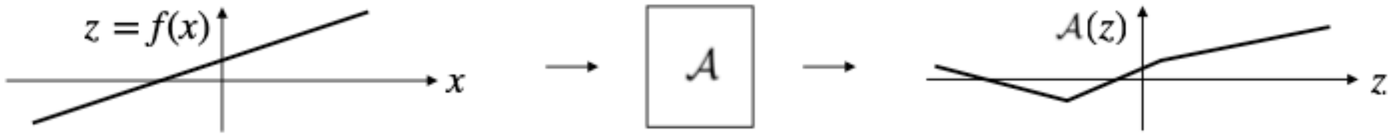


Activation and Nonlinearity

- Activations add nonlinearity so networks can model complex functions.
- Efficient form: $A(W \cdot o_{l-1} + b)$ using matrix multiplications.
- Different activations have different ranges and behaviors.

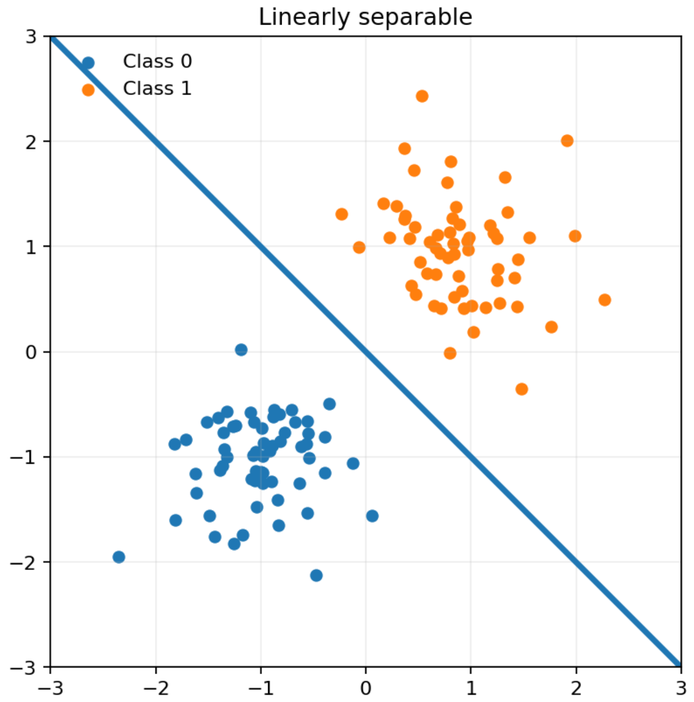


- *Activation function* $\mathcal{A}$: This is a function applied to the result of the weight multiplication and bias addition, and aims at introducing nonlinearities into the model.



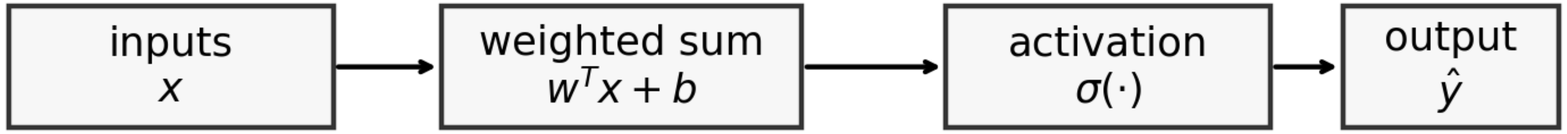
Why linear models?

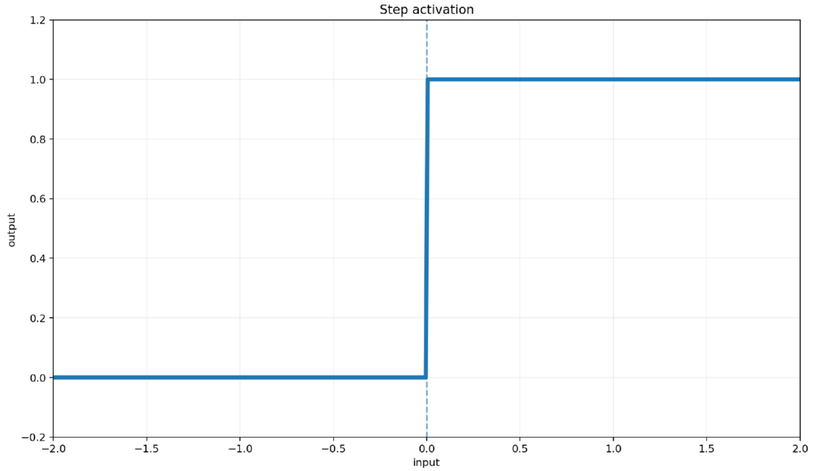
- A single line can only separate “linearly separable” datasets.
  - A linear model separates classes with a single line or hyperplane.
  - This limitation motivates perceptrons, hidden layers, and MLPs.



Perceptron : Component

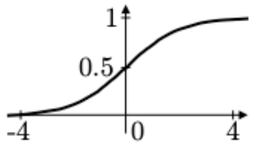
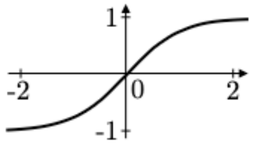
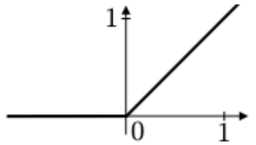
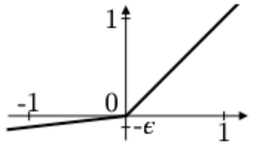
- Key Idea : A linear Score + A nonlinearity
  - The perceptron first computes a linear score $w^T x + b$.
  - An activation function converts that score into a decision.
  - This unit is the basic building block of a neural network.

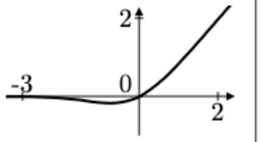
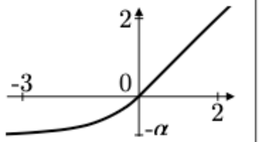




Activation and Nonlinearity

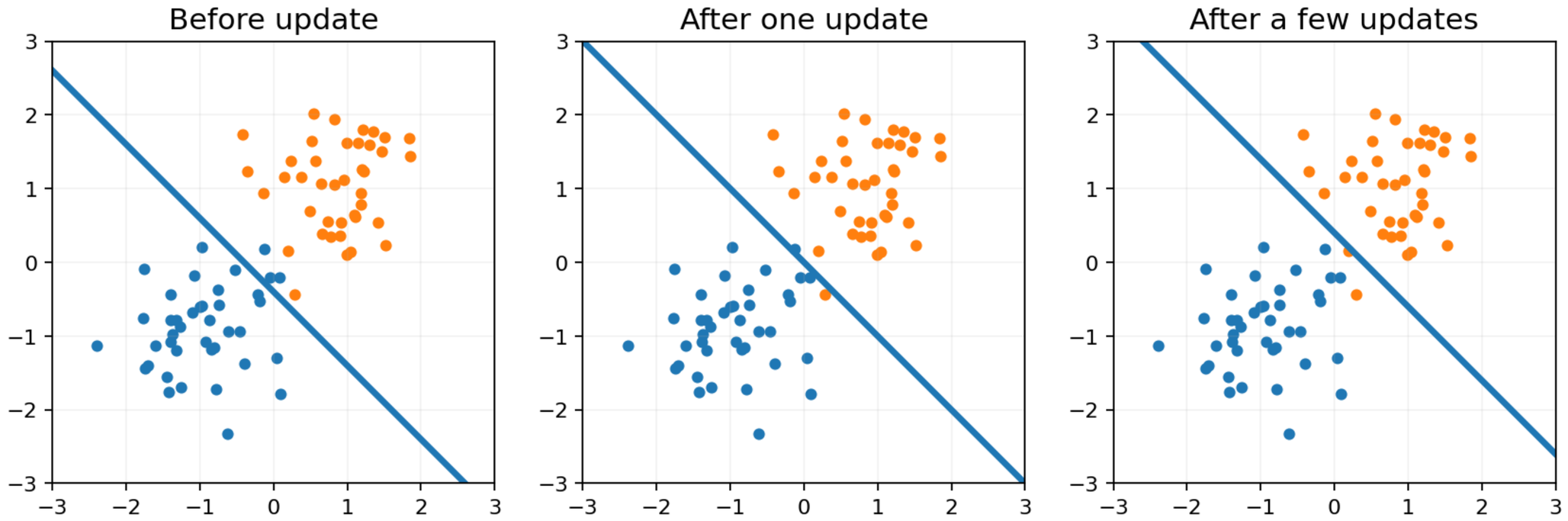
- Sigmoid/Tanh are bounded; useful for probabilities or normalized signals.
- ReLU-family is simple and often trains faster in deep networks.
- Choose based on curve shape, output range, and layer role.

Name	Function $\mathcal{A}(z)$	Output range	Illustration
Sigmoid σ *Sigmoid function*	$\frac{1}{1+e^{-z}}$	$]0,1[$	
Tanh *Hyperbolic Tangent*	$\frac{e^z-e^{-z}}{e^z+e^{-z}}$	$]-1,1[$	
ReLU *Rectified Linear Unit*	$\max(0,z)$	$[0,+\infty[$	
Leaky ReLU *Leaky Rectified Linear Unit*	$\max(\epsilon z, z)$ with $\epsilon \ll 1$	$]-\infty,+\infty[$	

Name	Function $\mathcal{A}(z)$	Output range	Illustration
GELU *Gaussian Error Linear Unit*	$zP(X \leqslant z)$ with $X \sim \mathcal{N}(0,1)$	$]-0.17,+\infty[$	
ELU *Exponential Linear Unit*	$\max(\alpha(e^z-1),z)$ with $\alpha \approx 1$	$]-\alpha,+\infty[$	

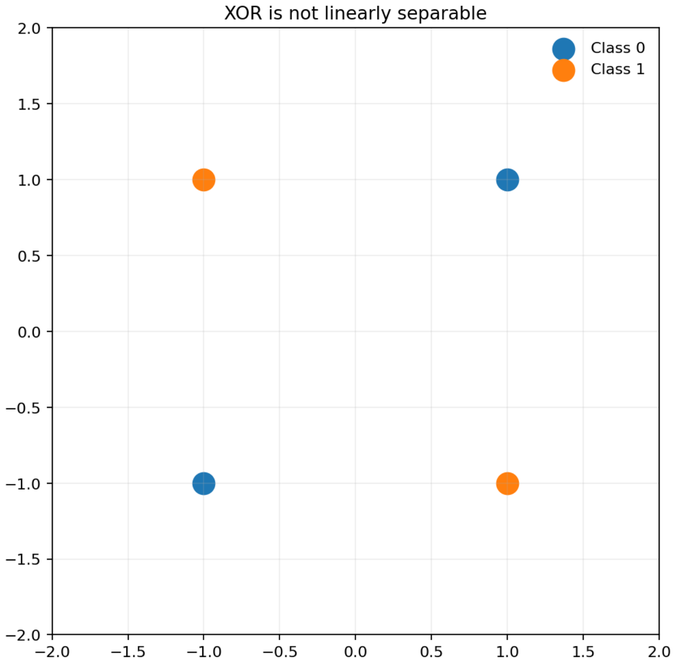
Perceptron update moves the boundary

- The perceptron updates its weights only when it makes a mistake.
  - Each misclassified example pushes the boundary in a corrective direction.
  - Repeated updates gradually align the separator with the data.

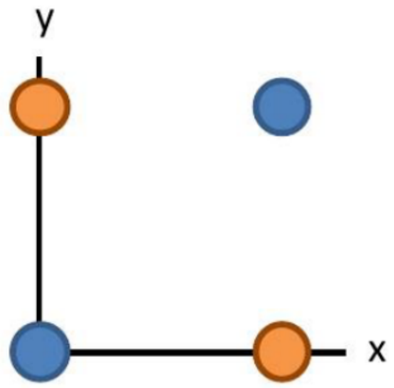


XOR problem

- Minsky and Papert (1969) famously analyzed limitations of single-layer perceptrons.
- XOR example showcases that some functions require multiple layers or nonlinearity.
- This moment contributed to skepticism and a slowdown in neural network research.



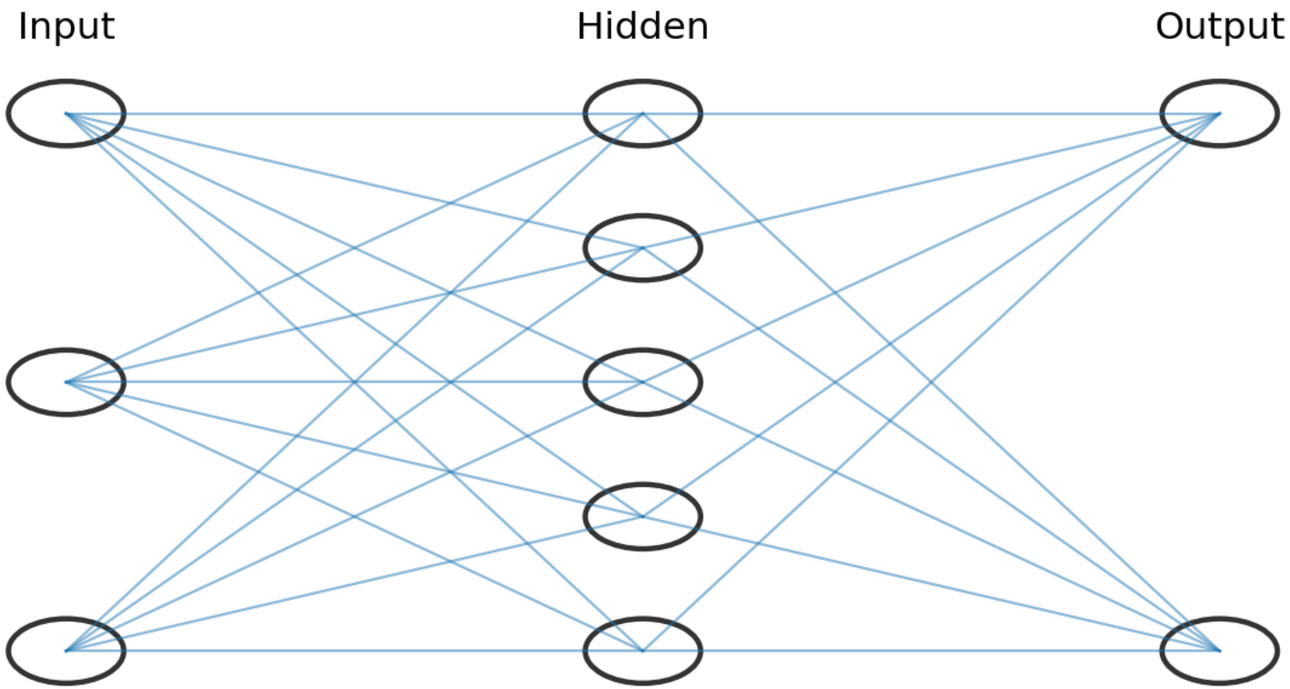
X	Y	F(x,y)
0	0	0
0	1	1
1	0	1
1	1	0



Showed that Perceptrons could not learn the XOR function
Caused a lot of disillusionment in the field

Perceptron → MLP

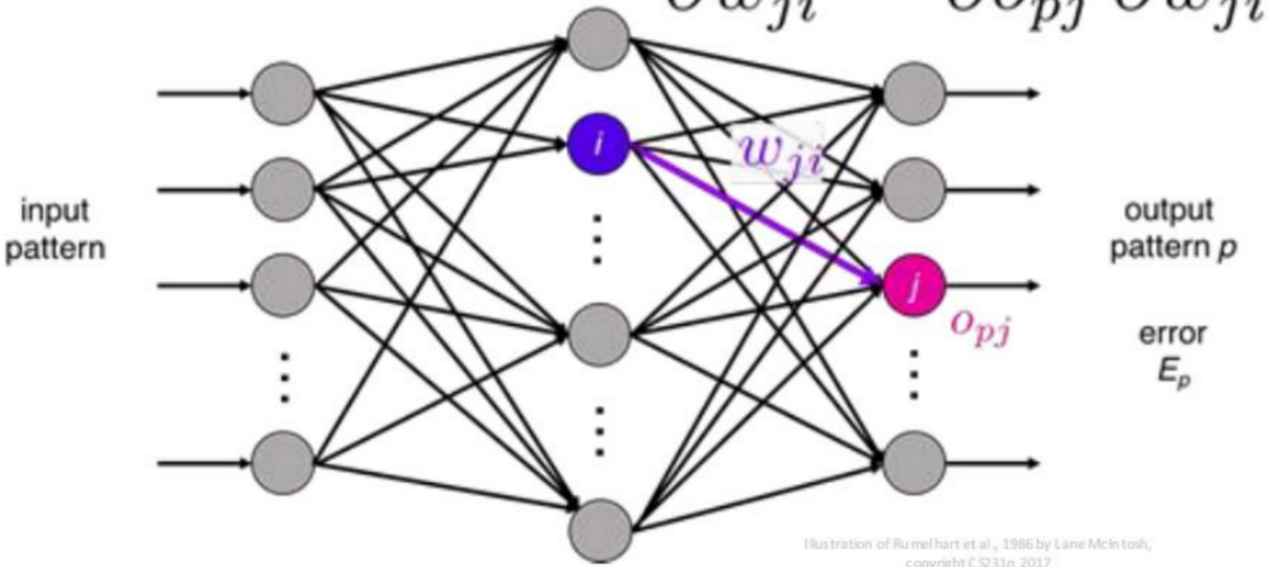
- Nonlinearity in Hidden layers enables non-linear decision boundaries.
- Multi-Layer Perceptron (MLP) : Add Hidden Layers



Backprop: Rumelhart, Hinton, and Williams, 1986

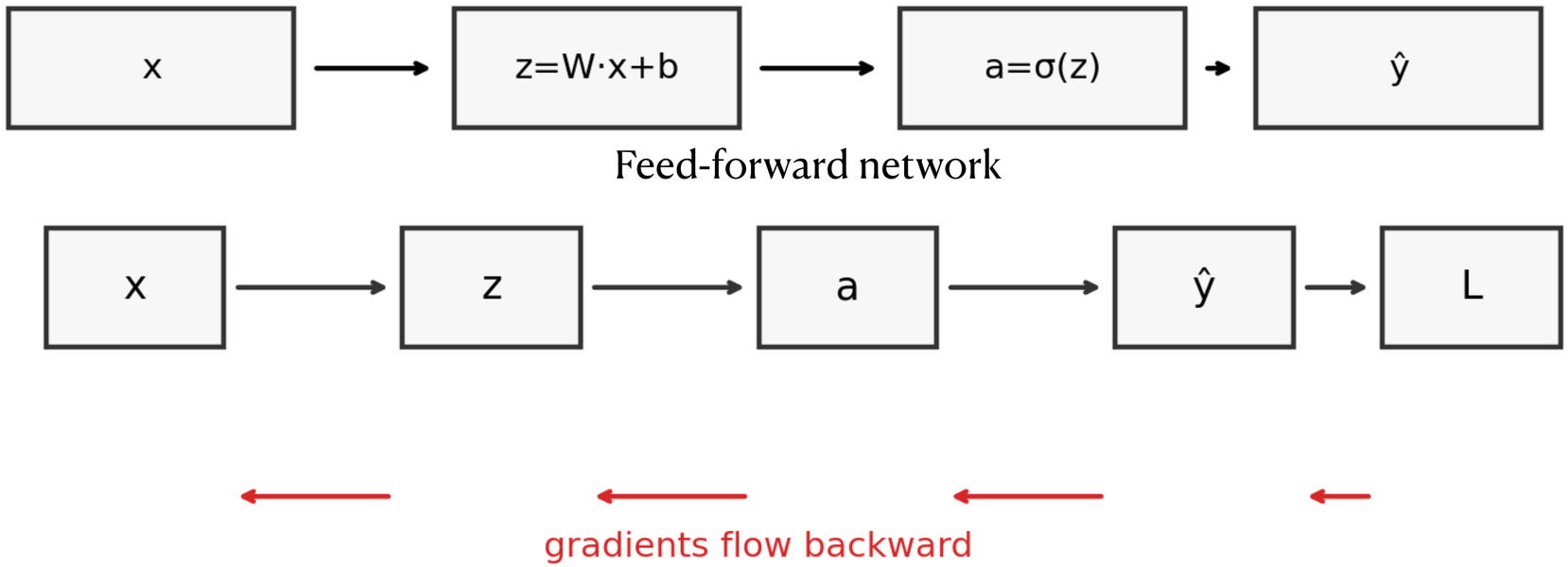
- Backpropagation: a practical way to compute gradients in multi-layer networks.
  - We can optimize deep models using data rather than manual tuning.
  - This is the algorithmic foundation behind modern deep learning training.

$$\frac{\partial E_p}{\partial w_{ji}} = \frac{\partial E_p}{\partial o_{pj}} \frac{\partial o_{pj}}{\partial w_{ji}}$$



Backprop: Rumelhart, Hinton, and Williams, 1986

- Forward Pass as a computation graph
- Backprop computes gradients by running the graph backward (chain rule)
  - Compute gradients efficiently for all parameters.
- A neural network is a differentiable program.



Chain Rule (One Picture)

- Backpropagation is the chain rule applied repeatedly through the network.
- Each local derivative measures how a small change affects the next variable.
- Multiplying these local terms gives gradients for earlier layers.

$$x \to z \to y$$

$$\frac{dy}{dx} = \frac{dy}{dz} \cdot \frac{dz}{dx}$$

Backprop is chain rule applied repeatedly on a graph.

Framing : Architecture, Training

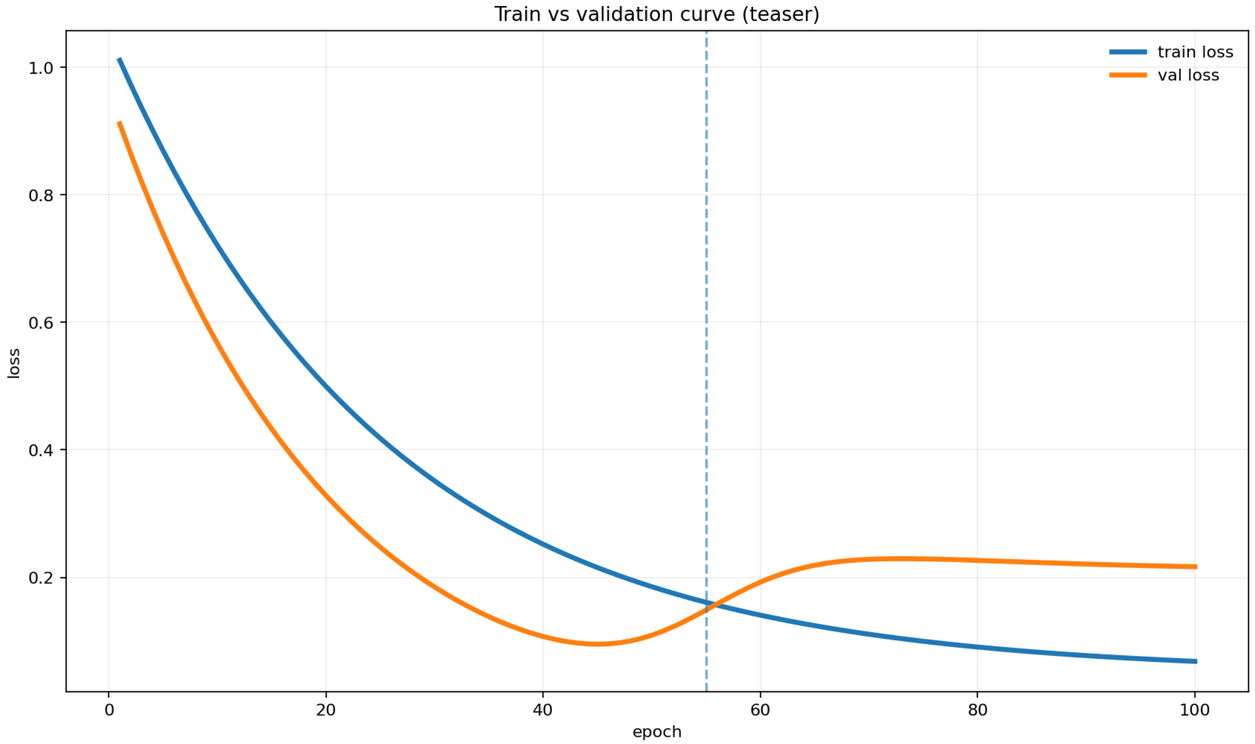
- We study each model through architecture, data & training, and domain extension.
- Architecture defines what kinds of functions the model can represent.
- Training determines what the model actually learns from data.

K-fold cross validation

- K-fold cross validation splits the dataset into K disjoint folds
- Each fold is used once for validation while the others are used for training.
- Averaging across folds gives a more reliable estimate when data is limited.

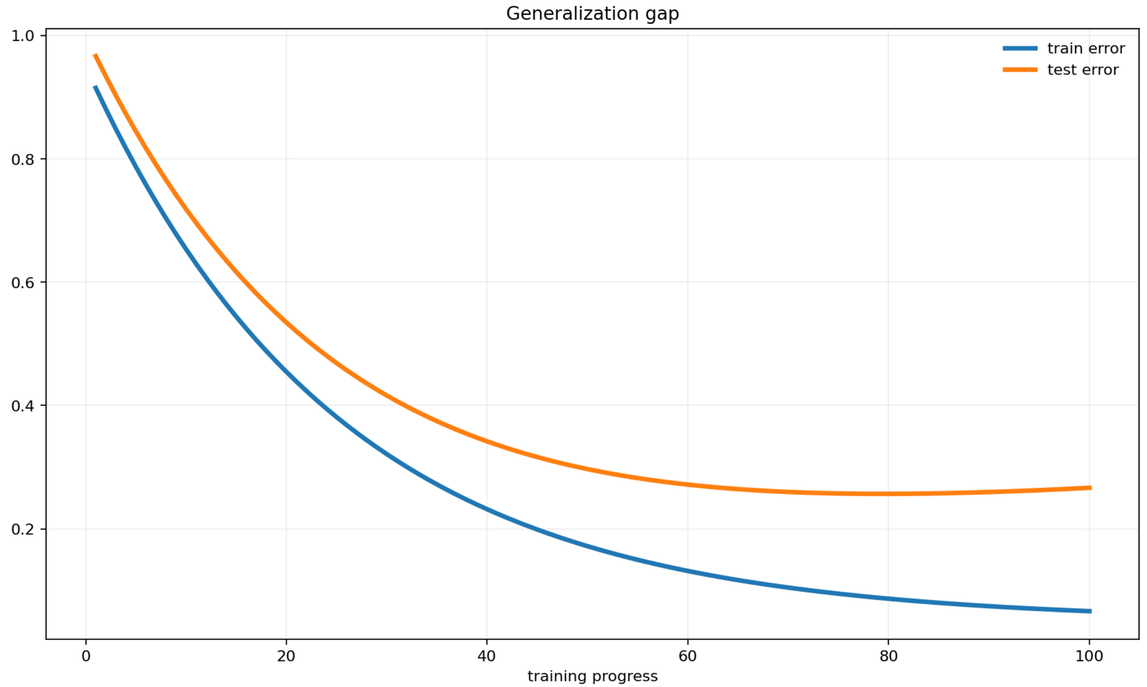
Expressivity vs Generalization

- Expressivity is the ability to fit richer functions and more complex patterns.
- Generalization asks whether those patterns still hold on unseen data.
- A more expressive model can lower training loss while hurting validation loss.



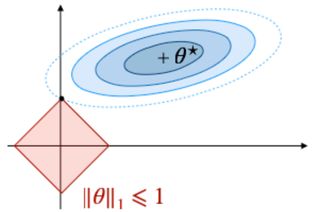
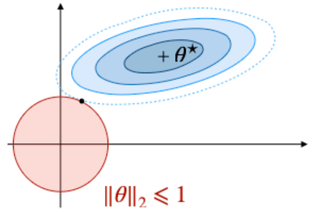
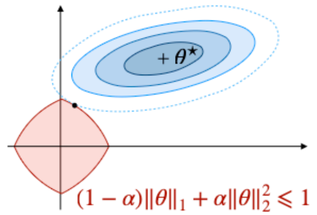
Generalization gap

- The generalization gap is the difference between training error and test error.
- A small gap suggests stable learning, while a large gap signals overfitting.
- This gap is why we use validation, regularization, and early stopping.



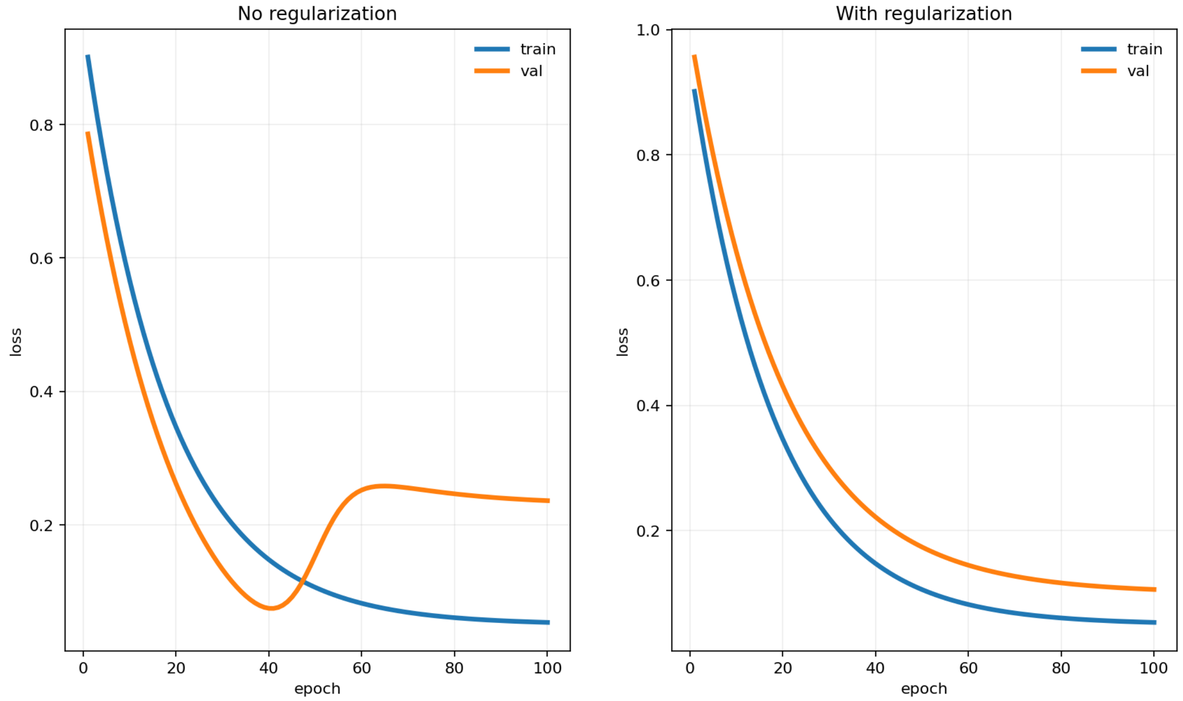
Weight Regularization

- L1 (LASSO): encourages sparsity; can drive some weights to zero.
- L2 (Ridge): shrinks weights smoothly; reduces overfitting.
- Elastic Net : Trade-off between variable selection and small coefficients

Regularizer	Description	Illustration
***L**east **A**bsolute **S**hrinkage and **S**election **O**perator* (LASSO)	$\mathscr{L} + \lambda\|\|\theta\|\|_1$ with $\lambda > 0$ • Shrinks coefficients to 0 • Good for variable selection • L_1 regularization	
Ridge	$\mathscr{L} + \lambda\|\|\theta\|\|_2^2$ with $\lambda > 0$ • Reduces magnitude of coefficients • L_2 regularization	
Elastic Net	$\mathscr{L} + \lambda\left[(1-\alpha)\|\|\theta\|\|_1 + \alpha\|\|\theta\|\|_2^2\right]$ with $\lambda > 0$ and $\alpha \in [0,1]$ Trade-off between variable selection and small coefficients that can be tuned with α	

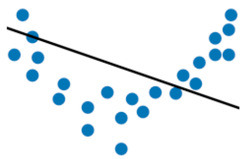
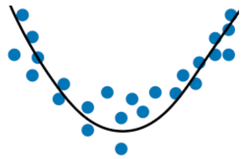
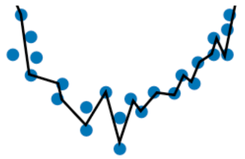
Case study: Regularization on/off

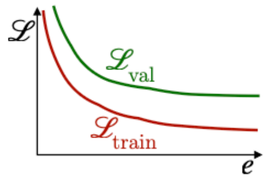
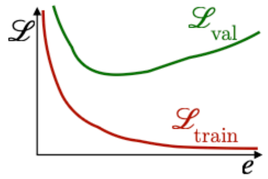
- Without regularization, validation loss rises after early improvement.
- With regularization, the model gives up a little fit to gain much better stability.
- Better generalization matters more than the lowest possible training loss.



Bias-Variance Trade-Off

- High bias → too simple → underfitting.
- High variance → too sensitive → overfitting.
- Typical fixes: adjust model size, regularize, get more data, or train longer.

	Underfitting	Just right	Overfitting
Symptoms	• High training error • Training error close to test error • High bias	Training error slightly lower than test error	• Low training error • Training error much lower than test error • High variance
Intuition			
Regression			

	Underfitting	Just right	Overfitting
Evolution of the loss			
Possible remedies	• Complexify model • Add more features • Train longer		• Regularize • Get more data

Takeaway

- MLPs become powerful because hidden layers and nonlinear activations can represent patterns that a single perceptron cannot.

- Backpropagation made multilayer networks practical by computing gradients efficiently and training the whole model end to end.

- Good learning is not just fitting the training set; it is balancing expressivity, regularization, and the bias-variance trade-off to generalize well.

Too Much Information : Bias-Variance Decomposition

1. Step 1 : Plug $Y = f(x) + \varepsilon$ into $\mathscr{E}(x)$
    1. Suppose to have $Y = f(x) + \varepsilon,\ E[\varepsilon \mid x] = 0,\ Var(\varepsilon \mid x) = \sigma^2$
    2. Set $\hat{f}_D(x)$ as the learned model and look into $\mathscr{E}(x) = \mathbb{E}_D\left[(Y - \hat{f}_D(x))^2 \mid x\right]$
    3. $\mathscr{E}(x) = \mathbb{E}_D\left[(Y - \hat{f}_D(x))^2 \mid x\right]$ can be replaced with $\mathbb{E}_D\left[(f(x) - \hat{f}_D(x))^2 + 2\varepsilon(f(x) - \hat{f}_D(x)) + \varepsilon^2\right]$
    4. It's also replaced with $\mathbb{E}_D\left[(f(x) - \hat{f}_D(x))^2\right] + 2\mathbb{E}_D\left[\varepsilon(f(x) - \hat{f}_D(x))\right] + \mathbb{E}[\varepsilon^2]$
2. Step 2 : Cross term
    1. When set $\mathbb{E}[\varepsilon \mid x] = 0$ and ε independent from D and randomness, get $\mathbb{E}_{D,\varepsilon}\left[\varepsilon(f(x) - \hat{f}_D(x))\right] = 0\,,\ \mathbb{E}[\varepsilon^2] = \sigma^2$
    2. $\mathscr{E}(x) = \mathbb{E}_D\left[(f(x) - \hat{f}_D(x))^2\right] + \sigma^2$ we call σ^2 as irreducible noise.

Too Much Information : Bias-Variance Decomposition

3. Step 3. Expected predictor
    1. We call $\bar{f}(x) = \mathbb{E}_D[\hat{f}_D(x)]$ as expected predictor
    2. $f(x) - \hat{f}_D(x) = \left(f(x) - \bar{f}(x)\right) + \left(\bar{f}(x) - \hat{f}_D(x)\right)$ comes out.
4. Step 4. Expand the equation again.
    1. We get $\mathbb{E}_D\left[(f(x) - \hat{f}_D(x))^2\right] = \mathbb{E}_D\left[\left(f(x) - \bar{f}(x) + \bar{f}(x) - \hat{f}_D(x)\right)^2\right]$

$$= \mathbb{E}_D\left[(f(x) - \bar{f}(x))^2\right] + \mathbb{E}_D\left[(\bar{f}(x) - \hat{f}_D(x))^2\right] + 2\mathbb{E}_D\left[(f(x) - \bar{f}(x))(\bar{f}(x) - \hat{f}_D(x))\right]$$

    2. $(f(x) - \bar{f}(x))^2 + \mathbb{E}_D\left[(\hat{f}_D(x) - \bar{f}(x))^2\right] + 2(f(x) - \bar{f}(x))\mathbb{E}_D[\bar{f}(x) - \hat{f}_D(x)]$ because $f(x) - \bar{f}(x)$ becomes constant w.r.t D
    3. $\mathbb{E}_D[\bar{f}(x) - \hat{f}_D(x)] = \bar{f}(x) - \mathbb{E}_D[\hat{f}_D(x)] = 0$, so it leads cross term equals to 0
    4. $\mathbb{E}_D\left[(f(x) - \hat{f}_D(x))^2\right] = (f(x) - \bar{f}(x))^2 + \mathbb{E}_D\left[(\hat{f}_D(x) - \bar{f}(x))^2\right]$

Too Much Information : Bias-Variance Decomposition

5. Step 5. Name “Bias" and “Variance”
    1. We set ‘Bias’ as
    $\text{Bias}(x) = \bar{f}(x) - f(x) = \mathbb{E}_D[\hat{f}_D(x)] - f(x) \text{Var}(x) = \mathbb{E}_D\left[(\hat{f}_D(x) - \bar{f}(x))^2\right]$
    2. Finally, we get

$$\mathscr{E}(x) = \underbrace{\text{Bias}(x)^2}_{\text{systematic error}} + \underbrace{\text{Var}(x)}_{\text{sensitivity to data}} + \underbrace{\sigma^2}_{\text{irreducible noise}}$$

$$\mathbb{E}_{D,\varepsilon}\left[(Y - \hat{f}_D(x))^2 \mid x\right] = \text{Bias}(x)^2 + \text{Var}(x) + \sigma^2$$

Q&A

Any question?